THE POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY 1

Final Report

BUSINESS INTELLIGENCE

BI System for E-Commerce Data Analysis

Group: 7

Class: E22HTTT

Member: NGUYỄN VIỆT HOÀNG - B22DCVT214

NGUYỄN BÌNH MINH - B22DCDT192

Hà Nội - 2025

Chapter 1: Introduction

1.1 Background and Context

The global e-commerce industry has undergone explosive growth over the past decade. According to industry reports, global e-commerce sales have surpassed several trillion US dollars annually, with developing markets such as Latin America experiencing particularly rapid expansion. In Brazil specifically, the e-commerce sector has grown significantly, driven by increasing internet penetration, mobile device adoption, and the emergence of large marketplace platforms connecting thousands of small and medium-sized businesses to consumers across the country.

This growth has produced an enormous volume of transactional data. Every customer visit, product search, order placement, payment, and delivery generates data points that, in aggregate, reflect the real-time health and dynamics of an e-commerce business. However, raw transactional data stored in operational databases is not directly useful for business analysis. It is fragmented across multiple tables, optimized for fast writes rather than fast reads, and lacks the aggregated, historical perspective that managers and analysts need to make informed decisions.

Business Intelligence (BI) addresses this challenge by providing a systematic approach to collecting, organizing, and analyzing business data. A well-implemented BI system transforms raw operational data into a structured, query-optimized data warehouse, and then surfaces insights through dashboards and reports. This allows decision-makers to monitor key performance indicators (KPIs), identify trends, and react to market changes based on evidence rather than intuition.

1.2 Problem Statement and Motivation

Despite the availability of transactional data, many e-commerce businesses — especially small and medium-sized ones — lack the infrastructure to turn that data into actionable insights. The raw data exists in flat files, spreadsheets, or operational databases, but without a properly designed data warehouse and visualization layer, answering even basic business questions becomes a manual and error-prone process.

Consider the following questions that any e-commerce business manager would want to answer regularly:

- Which products and product categories are generating the most revenue?

- How has monthly revenue trended over the past year, and are there seasonal patterns?
- Which regions of the country have the highest concentration of customers and sales?
- What proportion of customers make repeat purchases, and how does that compare to one-time buyers?

Answering these questions from raw transactional CSV files would require complex, time-consuming SQL queries or manual spreadsheet manipulation each time. A BI system with a proper data warehouse and dashboard makes these questions answerable in seconds, and keeps the answers up to date as new data arrives.

This project was motivated by the desire to build exactly such a system — starting from raw e-commerce data and ending with an interactive dashboard that makes key business metrics immediately accessible and explorable.

1.3 Objectives

The primary goal of this project is to design and implement a complete, end-to-end Business Intelligence system for e-commerce data analysis. This goal is broken down into the following specific objectives:

- Design a dimensional data warehouse using the Star Schema model, with a central fact table capturing order line items and four surrounding dimension tables representing customers, products, time, and geographic regions.
- Build an ETL (Extract, Transform, Load) pipeline in Python that reads raw CSV data from the Olist dataset, applies data cleaning and transformation logic, and loads the processed data into a PostgreSQL data warehouse.
- Deploy the PostgreSQL database and Metabase BI tool using Docker and Docker Compose, ensuring a reproducible and portable environment that can be set up consistently across different machines.
- Construct an interactive BI dashboard in Metabase that visualizes key business metrics including monthly revenue trends, top-selling products and categories, geographic distribution of sales, and customer purchase behavior.

- Derive and interpret meaningful business insights from the data, providing analysis that could inform real-world e-commerce decision-making.

1.4 Scope and Limitations

This project focuses on the analytical layer of a BI system. It does not involve building or modifying the operational systems that generate the data — the source data is treated as a static dataset obtained from Kaggle. The scope covers the full pipeline from raw CSV ingestion to dashboard visualization, but excludes real-time data streaming, automated data refresh scheduling, and machine learning or predictive analytics.

Several limitations of the dataset and implementation are worth noting:

- The dataset covers orders placed between 2016 and 2018. Data from 2016 is sparse, as Olist was in an early stage of growth, so trend analyses are most meaningful for the 2017–2018 period.
- The dataset has been anonymized. Customer names and seller identities are not available, limiting certain types of behavioral analysis.
- This project uses a static snapshot of the data. In a production BI system, the ETL pipeline would run on a schedule (e.g., daily) to keep the warehouse current. That scheduling layer is outside the scope of this academic project.

1.5 Tools and Technologies Used

The project was built entirely using open-source tools, all containerized with Docker for consistency and portability. The technology stack was chosen to reflect industry-standard BI tooling while remaining accessible for academic use.

PostgreSQL 15

PostgreSQL is the relational database management system used as the data warehouse. It is one of the most widely used open-source databases in the world, known for its robustness, standards compliance, and strong support for complex analytical queries. All dimension tables, the fact table, and the foreign key constraints are defined and managed in PostgreSQL. Analytical SQL queries used for insight generation are also executed against this database.

Python (Pandas, SQLAlchemy)

Python is used to implement the ETL pipeline. The Pandas library provides powerful data manipulation capabilities for reading CSV files, performing joins, deduplication, type conversion, and derived column computation. SQLAlchemy is used as the database interface layer to load the transformed DataFrames into PostgreSQL tables efficiently. Python was chosen because it offers a clean, readable way to express complex data transformation logic and has excellent library support for both data processing and database connectivity.

Metabase

Metabase is an open-source Business Intelligence and data visualization tool. It provides a web-based interface for connecting to databases, building queries visually or in SQL, and assembling interactive dashboards. Metabase was selected because it requires no licensing cost, is easy to self-host via Docker, and is capable of auto-detecting relationships in a star schema, making it straightforward to build multi-dimensional visualizations. Its simplicity makes it well-suited for academic BI projects while still reflecting real-world BI tool usage.

Docker and Docker Compose

Docker is used to containerize both PostgreSQL and Metabase. Docker Compose defines both services in a single docker-compose.yml file, allowing the entire environment — database server, BI tool, and their network connections — to be started with a single command (docker-compose up). This approach ensures that the project can be reproduced on any machine with Docker installed, regardless of the host operating system or pre-installed software. It also reflects the modern industry practice of deploying data infrastructure in containerized environments.

1.6 Report Structure

The remainder of this report is organized as follows:

- Chapter 2 describes the dataset used in this project — its source, structure, the individual CSV files, key attributes, and data quality challenges observed.
- Chapter 3 presents the Data Warehouse design, including the theoretical background of dimensional modeling, the Star Schema structure, and detailed definitions of all dimension and fact tables.

- Chapter 4 details the ETL pipeline implementation, covering each phase of the Extract, Transform, and Load process with specific reference to the Python code.
- Chapter 5 explains the dashboard construction in Metabase, including the Docker setup, database connection, and description of the key visualizations built.
- Chapter 6 analyzes the business insights derived from the data, covering revenue trends, geographic distribution of sales, customer purchase behavior, and top product performance.
- Chapter 7 concludes the report with a summary of the work completed, challenges encountered during the project, and suggestions for future improvements.

Chapter 2: Dataset Description

2.1 Data Source

The dataset used in this project is the Brazilian E-Commerce Public Dataset published by Olist, available on Kaggle. Olist is a Brazilian e-commerce platform that operates as a marketplace, connecting thousands of small and medium-sized merchants to customers across Brazil's major online retail channels. Rather than running its own storefront, Olist enables sellers to list their products on multiple marketplaces simultaneously through a single integrated platform.

The dataset is a publicly released, anonymized collection of real commercial data from the Olist platform. It contains information on approximately 100,000 orders placed between January 2016 and August 2018, spanning multiple Brazilian states and covering a wide range of product categories. The dataset captures each order from multiple perspectives: order status and timing, pricing and freight, customer location, product characteristics, and post-purchase customer reviews.

Because the data is real and anonymized (rather than synthetically generated), it contains the kinds of messiness and edge cases found in genuine operational data — including duplicate entries, missing values, and referential inconsistencies. This makes it particularly well-suited for a BI project that involves real ETL work, as opposed to a pre-cleaned academic dataset.

The data is made available by Olist under a Creative Commons license and is freely downloadable from the Kaggle platform. It has been widely used in the data science and analytics community for exploratory analysis, machine learning, and BI projects.

2.2 Dataset Structure and File Overview

The full Olist dataset consists of nine CSV files, each representing a different entity or relationship within the e-commerce system. The files are connected through shared identifiers, forming a relational structure that mirrors a typical operational database schema.

For this project, six of the nine files were selected based on their relevance to the defined Star Schema and the business questions being analyzed. The three files excluded from this project — olist_order_payments_dataset.csv, olist_order_reviews_dataset.csv, and olist_sellers_dataset.csv — contain payment method details, customer review text and scores, and seller information respectively. While these could support additional analyses,

they fall outside the scope of the four required dimensions (Customer, Product, Time, Region) and the three required dashboard metrics.

The six files used in this project are described below:

File Name	Description
olist_orders_dataset.csv	Core order records: order ID, customer ID, purchase timestamp, order status, and delivery timestamps.
olist_order_items_dataset.csv	Line-item details for each order: product ID, seller ID, price, freight value, and shipping limit date.
olist_products_dataset.csv	Product catalog: category name (in Portuguese), physical dimensions, weight, and name/description length.
olist_customers_dataset.csv	Customer records: per-order customer ID, stable unique customer ID, city, state, and zip code prefix.
olist_geolocation_dataset.csv	Geographic reference: Brazilian zip code prefixes mapped to city, state, latitude, and longitude.
product_category_name_translation.csv	Translation table mapping Portuguese product category names to English equivalents.

Table 2.1 — CSV files used in this project

2.3 Entity Relationships in the Source Data

The source files are related to each other through shared key fields. Understanding these relationships is essential for designing the ETL pipeline and the data warehouse schema. The main relationships are:

- olist_orders_dataset links to olist_customers_dataset via customer_id. Each order belongs to exactly one customer.
- olist_order_items_dataset links to olist_orders_dataset via order_id. Each order can have one or more line items.

- olist_order_items_dataset links to olist_products_dataset via product_id. Each line item references one product.
- olist_customers_dataset links to olist_geolocation_dataset via customer_zip_code_prefix. Each customer maps to a geographic region.
- olist_products_dataset links to product_category_name_translation via product_category_name. Each product category has an English translation.

These relationships form the foundation of the data warehouse design. The ETL pipeline follows these joins to assemble dimension and fact tables from the source files.

2.4 Key Attributes and Business Meaning

Each file contributes specific fields that map to dimensions or measures in the data warehouse. The following describes the most analytically significant attributes used in this project, grouped by entity.

Orders and Order Items

- order_id — A unique identifier for each customer order. Used as a degenerate dimension in the fact table, allowing order-level grouping without a dedicated order dimension.
- order_purchase_timestamp — The exact date and time when the customer placed the order. This is the source field for the entire time dimension, from which year, month, quarter, day, and day of week are derived.
- order_status — The delivery status of the order (e.g., delivered, shipped, canceled). While not used as a dimension in this project, it was considered during data quality assessment; only delivered orders were included in revenue calculations.
- price — The unit selling price of a product in a given order line, in Brazilian Reais (BRL). This is the primary revenue measure stored in the fact table.
- freight_value — The shipping cost charged for a given order line. Stored alongside price in the fact table to support logistics and cost analysis.

Customers and Geography

- customer_id — An order-scoped identifier that changes with each new order placed by the same person. This field is used to join orders to customers but is NOT used as the customer dimension key.
- customer_unique_id — A stable, person-level identifier that remains consistent across multiple orders. This is the natural key of the customer dimension and is the correct field for identifying repeat customers.
- customer_city / customer_state — The city and state associated with the customer's delivery address. Used as descriptive attributes in the customer dimension.
- customer_zip_code_prefix — A five-digit prefix that serves as the geographic link between customers and the region dimension.
- geolocation_lat / geolocation_lng — Latitude and longitude coordinates associated with a zip code prefix. Averaged across multiple entries per prefix and stored in dim_region to support map-based visualizations.

Products

- product_id — Unique identifier for each product, used as the natural key of the item dimension.
- product_category_name — Product category in Portuguese. Joined with the translation table during ETL to produce the English equivalent stored in dim_item.
- product_weight_g — Product weight in grams. Stored in the item dimension for potential logistics analysis.
- product_length_cm, product_height_cm, product_width_cm — Physical dimensions used to derive product volume (length × height × width), stored as volume_cm3 in dim_item.

2.5 Dataset Scale and Coverage

To give a sense of the dataset's scale and coverage, the following approximate figures are drawn from the Olist dataset:

- Approximately 100,000 orders recorded across the dataset's time range.
- Orders span from January 2016 to August 2018, covering a period of approximately 32 months.
- Customers are distributed across all 26 Brazilian states plus the Federal District, with the highest concentrations in São Paulo (SP), Rio de Janeiro (RJ), and Minas Gerais (MG).
- The product catalog covers over 70 distinct categories, ranging from electronics and furniture to health and beauty products.
- The geolocation dataset contains over 1,000 unique zip code prefixes with coordinate data.

This scale is sufficient to produce statistically meaningful business insights while remaining manageable for a local Docker-based data warehouse without requiring distributed computing infrastructure.

2.6 Data Quality Challenges

A thorough review of the raw data prior to ETL design revealed several data quality issues that needed to be addressed. Understanding these challenges is important context for the transformation decisions described in Chapter 4.

Dual Customer Identifiers

The customers dataset contains two distinct identifier fields: customer_id (unique per order) and customer_unique_id (unique per real person). A customer who places three orders will appear three times in the dataset with three different customer_id values but the same customer_unique_id. If the dimension table were built using customer_id as the key, repeat customers would be counted as multiple distinct individuals. This would distort customer count metrics and make it impossible to identify returning buyers. The ETL pipeline resolves this by deduplicating on customer_unique_id before loading dim_customer.

Multiple Geolocation Entries per Zip Code

The geolocation dataset maps zip code prefixes to coordinates, but each prefix appears multiple times with slightly different latitude/longitude values — reflecting data collected from different sources or at different times. Using any single entry arbitrarily would introduce noise; creating multiple region rows per zip code would make the join to

customers ambiguous. The solution adopted is to compute the mean latitude and mean longitude across all entries for each zip code prefix, yielding a single, representative coordinate per region.

Orphan Records and Missing Foreign Keys

Some order items reference product IDs that do not appear in the products dataset, and some customer zip codes do not have corresponding entries in the geolocation dataset. These orphan records would violate the foreign key constraints defined in the data warehouse schema if loaded without filtering. The ETL pipeline identifies and removes these records during the Transform phase, ensuring that only fact rows with resolvable dimension keys are loaded into the warehouse.

Portuguese Category Names

All product category names in olist_products_dataset.csv are in Portuguese (e.g., "cama_mesa_banho" for bed, table, and bath products). Without translation, these labels would appear in their Portuguese form on the dashboard, making them difficult to interpret. The product_category_name_translation.csv file provides English equivalents for all categories, and this translation is applied as a JOIN during the ETL Transform phase so that dim_item stores English category names directly.

Sparse Data in 2016

The dataset's earliest orders date from January 2016, but the volume of orders in 2016 is significantly lower than in 2017 or 2018. This reflects Olist's early-stage growth during that period rather than a data collection gap. As a result, time-series analyses — particularly monthly revenue trends — are most meaningful when focused on 2017 onwards. This limitation is acknowledged in the insight analysis chapter.

Chapter 3: Data Warehouse Design

3.1 Theoretical Background

A Data Warehouse (DW) is a centralized, subject-oriented, integrated, and time-variant repository of data designed specifically to support analytical decision-making. Unlike an operational database, which is optimized for transactional processing (OLTP — Online Transaction Processing), a data warehouse is structured for analytical workloads (OLAP — Online Analytical Processing). The key differences are important to understand before discussing the design choices made in this project.

In an OLTP system, the primary goal is to record business events quickly and accurately: a customer places an order, a payment is processed, a product is shipped. These systems are highly normalized to minimize data redundancy and ensure fast write performance. However, answering complex analytical questions — such as "what was total revenue by state for each month of 2018?" — on a normalized OLTP schema requires many expensive JOIN operations across multiple tables, making it poorly suited for BI.

A Data Warehouse solves this by restructuring data into a format optimized for reading and aggregation. Data is extracted from one or more source systems, cleaned and transformed, and loaded into the warehouse in a denormalized, multidimensional structure. Once in the warehouse, business users and BI tools can query large volumes of historical data efficiently, without impacting the performance of operational systems.

For this project, the data warehouse is built on PostgreSQL and populated from the Olist CSV dataset. All analytical queries and dashboard visualizations are performed against the warehouse, not the raw CSV files.

3.1.1 Dimensional Modeling and the Star Schema

Dimensional modeling, introduced by Ralph Kimball, is the dominant methodology for designing data warehouses intended for BI use. It organizes data into two types of tables: Fact Tables and Dimension Tables.

- A Fact Table stores quantitative, measurable business events — typically numeric values such as revenue, quantity, or cost. Each row in a fact table represents a single business event at a defined level of granularity (the "grain").

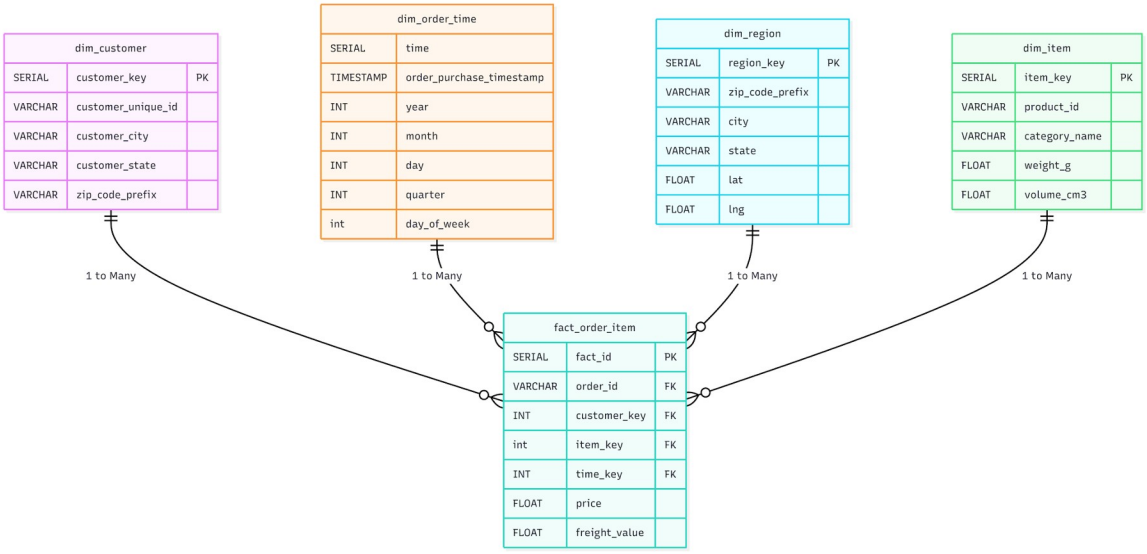
- Dimension Tables provide descriptive context for the facts — the who, what, when, and where of each event. They are typically wider tables with textual attributes used for filtering, grouping, and labeling results in reports.

The Star Schema is the simplest and most common dimensional model. It places a single fact table at the center, with dimension tables radiating outward — visually resembling a star. This model was chosen for this project because:

- It produces queries that are simple to write and easy for BI tools to generate automatically, since each dimension requires only a single JOIN to the fact table.
- It performs well at scale because denormalization reduces the number of joins required during analytical queries.
- It aligns naturally with the four required dimensions specified in the project requirements: Customer, Product, Time, and Region.
- It is well-supported by Metabase, which can auto-detect star schema relationships and generate visual queries without writing SQL manually.

3.2 Schema Overview

The data warehouse in this project consists of five tables: one central Fact Table (fact_order_item) and four surrounding Dimension Tables (dim_customer, dim_region, dim_item, and dim_order_time). Together, these tables form the Star Schema illustrated below.



Each dimension table connects to the fact table through a surrogate integer foreign key. The relationships are all one-to-many: a single customer can appear in many fact rows (one per order item purchased), a single region can be associated with many customers and orders, a single product can appear in many order items, and a single time record corresponds to many order events that share the same purchase timestamp.

This structure means that any analytical query — regardless of which dimension it slices by — only needs to join the fact table with at most one or two dimension tables, keeping query complexity low and execution fast.

3.3 Dimension Tables

The four dimension tables correspond directly to the four analytical perspectives required by the project: Customer, Region (geographic), Item (product), and Time. Each dimension table uses a surrogate primary key — an auto-incremented integer generated by the warehouse — rather than the natural key from the source data. This design choice

decouples the warehouse from the source system and ensures stable, consistent relationships even if source identifiers change.

3.3.1 dim_customer

The customer dimension stores one record per unique real-world customer. An important distinction exists in the source data: the olist_customers_dataset contains two customer identifier fields — customer_id, which is unique per order, and customer_unique_id, which is stable across a customer's lifetime and represents a single real individual. Using customer_id as the dimension key would cause repeat customers to appear as multiple distinct people, distorting customer behavior analysis. Therefore, dim_customer is deduplicated on customer_unique_id, ensuring each real customer appears exactly once.

Column	Data Type	Description
customer_key	SERIAL	Surrogate primary key, auto-generated by the warehouse.
customer_unique_id	VARCHAR	Natural key from source data. Stable identifier representing one real customer across multiple orders.
customer_city	VARCHAR	City where the customer is located.
customer_state	VARCHAR	Brazilian state abbreviation (e.g. SP, RJ, MG).
zip_code_prefix	VARCHAR	Five-digit zip prefix. Used to join with dim_region for geographic analysis.

Table 3.1 — dim_customer

This dimension enables analyses such as identifying which states generate the most customers, determining whether a customer made a single purchase or returned for multiple orders, and segmenting customers geographically. The zip_code_prefix field serves as a soft link to dim_region, allowing the customer's location to be cross-referenced with full geographic coordinates.

3.3.2 dim_region

The region dimension provides geographic reference data at the zip code prefix level. Brazil uses a five-digit zip code system (CEP), and Olist's geolocation dataset maps each zip code prefix to a city, state, latitude, and longitude. Since the geolocation dataset contains multiple coordinate entries per zip prefix (due to multiple data sources), these are averaged during the ETL process to produce a single, representative latitude and longitude per region. This averaging approach is intentional — it provides a good central point for map visualizations without the complexity of handling multiple coordinates per zip.

Column	Data Type	Description
region_key	SERIAL	Surrogate primary key.
zip_code_prefix	VARCHAR	Natural key. Five-digit Brazilian zip prefix from the geolocation dataset.
city	VARCHAR	City name associated with this zip code prefix.
state	VARCHAR	Brazilian state abbreviation.
lat	FLOAT	Average latitude of all geolocation entries for this zip prefix.
lng	FLOAT	Average longitude of all geolocation entries for this zip prefix.

Table 3.2 — dim_region

This dimension is what enables geographic analysis of sales data. By joining fact_order_item to dim_region through dim_customer, it becomes possible to visualize total revenue or order volume by city, state, or specific geographic coordinates. In Metabase, the lat and lng fields can be used directly to render pin maps showing where orders are concentrated across Brazil.

3.3.3 dim_item

The item dimension represents the product catalog. In the source data, product information is spread across two files: olist_products_dataset.csv, which contains product attributes in Portuguese, and product_category_name_translation.csv, which maps Portuguese category names to English. During the ETL process, these two files are joined

so that dim_item stores the English category name directly, eliminating the need for a separate translation join at query time.

Physical product attributes such as weight and volume are also included, as they may be relevant for logistics analysis (e.g., understanding whether heavier products incur higher freight costs). Volume is derived by multiplying the three dimension fields: product_length_cm × product_height_cm × product_width_cm.

Column	Data Type	Description
item_key	SERIAL	Surrogate primary key.
product_id	VARCHAR	Natural key. Original product identifier from the Olist products dataset.
category_name	VARCHAR	English product category name, obtained by joining with the translation file during ETL.
weight_g	FLOAT	Product weight in grams.
volume_cm3	FLOAT	Derived product volume in cubic centimetres (length × height × width).

Table 3.3 — dim_item

This dimension supports product-level analyses such as identifying top-selling categories by revenue, comparing average order value across product types, and examining whether physical characteristics of products correlate with freight costs. Category grouping in particular is one of the three primary business analyses required by the project.

3.3.4 dim_order_time

The time dimension is derived from the order_purchase_timestamp field in the orders dataset. Rather than storing just the raw timestamp in the fact table and parsing it at query time, the ETL pipeline pre-computes all relevant date components and stores them as separate integer columns in dim_order_time. This is a standard data warehousing best practice that significantly improves the performance and readability of time-based queries.

For example, a query such as "total revenue grouped by month and year" requires no date parsing functions when the year and month are already stored as integers. Similarly, grouping by quarter or day of week becomes a simple GROUP BY on pre-computed integer fields rather than a function applied to every row at runtime.

Column	Data Type	Description
time_key	SERIAL	Surrogate primary key.
order_purchase_timestamp	TIMESTAMP	Full purchase timestamp from the source orders table. Retained for reference and fine-grained filtering.
year	INT	Year of the order (e.g. 2016, 2017, 2018).
month	INT	Month number (1 = January, 12 = December).
day	INT	Day of the month (1–31).
quarter	INT	Quarter of the year (1–4), derived from the month.
day_of_week	INT	Day of the week as an integer (0 = Monday, 6 = Sunday), useful for weekday vs. weekend analysis.

Table 3.4 — dim_order_time

This dimension directly enables the monthly revenue trend analysis required by the project, as well as any seasonal or weekly analyses. Metabase can use the year and month fields to generate time-series charts without needing to configure date extraction expressions manually.

3.4 Fact Table: fact_order_item

3.4.1 Defining the Grain

The most important design decision for any fact table is choosing its grain — the level of detail represented by a single row. A coarser grain (e.g., one row per order) is simpler but loses detail; a finer grain (e.g., one row per order item) retains full detail but produces a larger table.

For this project, the grain of fact_order_item is one row per order line item — that is, one row for each individual product purchased within an order. This grain was chosen because:

- It is the natural atomic unit captured in the source data (olist_order_items_dataset.csv stores one row per item).
- It preserves product-level detail, which is necessary for analyses like top-selling products or revenue per category.
- It supports aggregation up to any higher level — order, customer, region, month — without loss of information.

  As a result, a single order containing three different products would produce three separate rows in fact_order_item, each with its own product reference, price, and freight value. The order_id degenerate dimension is retained so that all items belonging to the same order can be grouped when needed.

3.4.2 Table Definition

Column	Data Type	Description
fact_id	SERIAL	Surrogate primary key of the fact table. Auto-incremented.
order_id	VARCHAR	Degenerate dimension. The original order identifier, stored directly in the fact table to allow order-level grouping without a separate order dimension.
customer_key	INT	Foreign key to dim_customer. Links this order item to the customer who placed the order.
region_key	INT	Foreign key to dim_region. Derived from the customer's zip code prefix, linking the order to a geographic location.
item_key	INT	Foreign key to dim_item. Identifies the specific product purchased in this order line.
time_key	INT	Foreign key to dim_order_time. Links the order item to its purchase date and time.

price	FLOAT	Unit price of the product in this order line (in Brazilian Reais). The primary revenue measure in the warehouse.
freight_value	FLOAT	Shipping cost for this order line. Enables analysis of logistics cost relative to product value.

Table 3.5 — fact_order_item

The two numeric measures — price and freight_value — are additive facts, meaning they can be meaningfully summed across any dimension. Total revenue for a month is the SUM(price) filtered by year and month. Total revenue for a state is the SUM(price) joined through dim_customer and dim_region. This additive property is what makes star schema fact tables so powerful for aggregation-heavy BI workloads.

3.5 Referential Integrity and Constraints

Referential integrity ensures that every foreign key value in the fact table has a corresponding primary key in the referenced dimension table. Without this guarantee, analytical queries could silently produce incorrect results — for example, aggregating revenue for order items whose customer or product no longer exists in the dimension tables.

In this project, foreign key constraints are defined in the PostgreSQL schema (init.sql) between fact_order_item and all four dimension tables. Specifically:

- fact_order_item.customer_key references dim_customer.customer_key
- fact_order_item.region_key references dim_region.region_key
- fact_order_item.item_key references dim_item.item_key
- fact_order_item.time_key references dim_order_time.time_key

These constraints are enforced at the database level by PostgreSQL, meaning any INSERT into the fact table that references a non-existent dimension key will be rejected. This makes referential integrity a property of the database itself, not just a convention maintained by the ETL code.

In practice, the ETL pipeline is designed to load all dimension tables before the fact table, and orphan records — fact rows whose dimension keys cannot be resolved — are filtered out during the Transform phase. This two-layer approach (ETL filtering + database constraints) ensures that the warehouse data is clean and consistent at all times.

3.6 Design Decisions and Justifications

Several deliberate design choices were made during the warehouse design phase. Each is explained below with its rationale.

Surrogate Keys over Natural Keys

All dimension tables use auto-incremented SERIAL integers (surrogate keys) as their primary keys, rather than the natural identifiers from the source data (such as product_id or customer_unique_id). Natural keys from operational systems can be long strings, may change over time, and are tightly coupled to the source system's design. Surrogate keys keep join columns small and fast (integer vs. VARCHAR comparisons), and insulate the warehouse from any future changes in source system identifiers.

Degenerate Dimension for order_id

The order_id field is stored directly in the fact table rather than in a separate order dimension table. In dimensional modeling, this is known as a degenerate dimension — a dimension attribute that has no associated descriptive data beyond the key itself. Creating a full order dimension table (with its own surrogate key) would add complexity and an extra JOIN without providing analytical value. Storing order_id directly in the fact table allows order-level grouping (e.g., calculating average items per order) while keeping the schema lean.

Averaged Geolocation Coordinates

The Olist geolocation dataset contains multiple latitude/longitude pairs for each zip code prefix, collected from different mapping sources. Rather than picking one arbitrarily or creating multiple region rows per zip code (which would complicate joins), the ETL pipeline computes the mean latitude and mean longitude across all entries for each zip prefix. The resulting single coordinate per region is accurate enough for city- and state-level map visualizations, which is the level of geographic granularity required by this project.

Denormalized English Category Names

Product category names in the Olist dataset are stored in Portuguese. Rather than keeping a separate translation lookup table in the warehouse, the translation is applied during the ETL Transform phase, and the English category name is stored directly in dim_item. This denormalization eliminates one JOIN from every product-category query and makes dashboard configurations simpler, since Metabase users see readable English labels without any additional mapping step.

Pre-computed Time Attributes

Date components (year, month, day, quarter, day_of_week) are extracted from the purchase timestamp and stored as integer columns in dim_order_time. The alternative — storing only the raw timestamp and using SQL date functions at query time — would work but adds computational overhead on every query and makes Metabase chart configuration less intuitive. Pre-computing these values follows the standard dimensional modeling practice of making the time dimension a "calendar table" that supports fast, readable time-based aggregations without date arithmetic in every query.

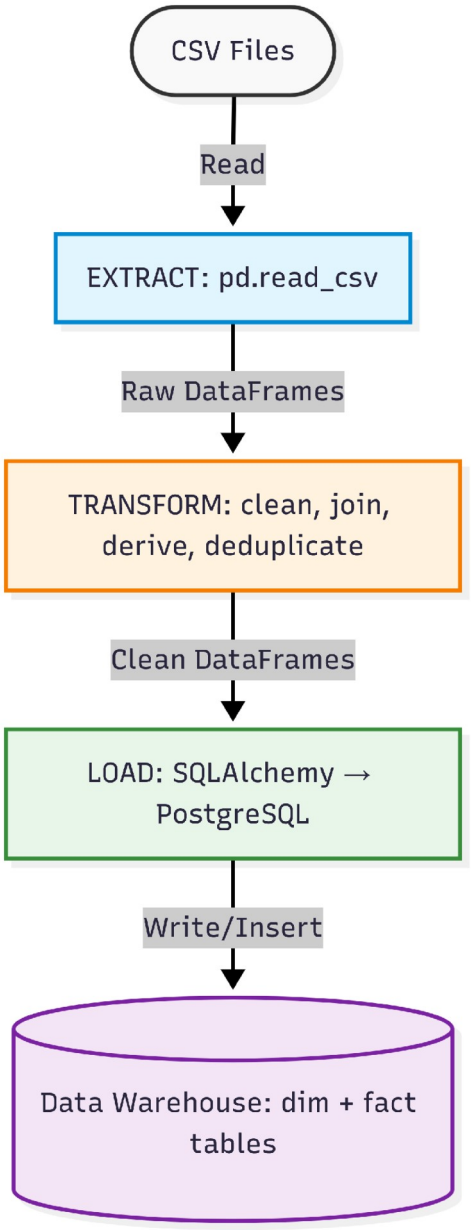
Chapter 4: ETL Pipeline

4.1 Overview of the ETL Process

ETL stands for Extract, Transform, and Load — the three fundamental phases of moving data from source systems into a data warehouse. It is the backbone of any Business Intelligence system, acting as the bridge between raw operational data and the clean, structured warehouse that BI tools query against.

In the Extract phase, data is read from one or more source systems — in this project, a collection of CSV files. In the Transform phase, the raw data is cleaned, restructured, enriched, and validated to conform to the data warehouse schema. This is typically the most complex phase, as it must handle all data quality issues, enforce business rules, and construct the dimension and fact table structures. In the Load phase, the transformed data is written into the target database tables in the correct order, respecting foreign key dependencies.

For this project, the ETL pipeline is implemented as a single Python script (etl_pipeline.py) using two core libraries: Pandas for data manipulation and SQLAlchemy for database connectivity. The pipeline is designed to be run once to perform a full historical load of all Olist data into the PostgreSQL data warehouse. The overall flow is illustrated below:



4.2 Environment and Dependencies

The ETL pipeline runs inside a Docker environment. The docker-compose.yml file defines two services: a PostgreSQL 15 database container and a Metabase container. The PostgreSQL container exposes port 5432 and is initialized on first startup using init.sql, which creates all dimension and fact tables along with their foreign key constraints. The Metabase container connects to the same PostgreSQL instance for dashboard queries.

The Python ETL script runs on the host machine (outside Docker) and connects to the PostgreSQL container via localhost:5432 using SQLAlchemy's connection string format. The required Python libraries are:

- pandas — for reading CSV files and all data transformation operations.
- sqlalchemy — for establishing the database connection and writing DataFrames to PostgreSQL tables.
- psycopg2 — the PostgreSQL driver used by SQLAlchemy under the hood.

The database schema (all CREATE TABLE statements with primary keys, foreign keys, and data types) is defined separately in init.sql and executed automatically when the PostgreSQL Docker container first starts. This separation of concerns — schema definition in SQL, data population in Python — keeps the pipeline clean and makes it easy to reset the database by simply recreating the container.

4.3 Extract Phase

The Extract phase reads all required source CSV files into memory as Pandas DataFrames. This is the simplest phase of the pipeline — its primary responsibility is to load the raw data exactly as it exists in the source files, without modification. Any errors at this stage (missing files, encoding issues, incorrect delimiters) would prevent the rest of the pipeline from running.

The six CSV files loaded during the Extract phase are:

- olist_orders_dataset.csv — loaded to provide the order_id, customer_id, and order_purchase_timestamp for each order.
- olist_order_items_dataset.csv — loaded to provide the product_id, price, and freight_value for each order line item.
- olist_products_dataset.csv — loaded to provide product category names and physical attributes.
- olist_customers_dataset.csv — loaded to provide the customer_unique_id, city, state, and zip code prefix for each customer.

- olist_geolocation_dataset.csv — loaded to provide latitude/longitude coordinates by zip code prefix.
- product_category_name_translation.csv — loaded to translate Portuguese category names to English.

All files are read using pd.read_csv() with UTF-8 encoding specified explicitly to avoid character encoding errors, particularly important for Portuguese text fields that may contain accented characters (e.g., São Paulo, Açaí). At this stage no filtering, renaming, or type conversion is applied — the DataFrames reflect the raw source data exactly.

One important consideration during extraction is memory management. The geolocation dataset in particular is large, containing over one million rows. However, since this dataset is processed only once (during dim_region construction) and the resulting dimension table has far fewer rows (one per unique zip prefix), the full geolocation DataFrame is only briefly held in memory before being aggregated and discarded.

4.4 Transform Phase

The Transform phase is the most complex and important part of the ETL pipeline. It applies all data cleaning, business logic, structural transformation, and validation needed to convert the raw source DataFrames into the final dimension and fact table structures defined in the data warehouse schema. The Transform phase is executed separately for each dimension table and then for the fact table. Each transformation is described in detail below.

4.4.1 Transforming dim_customer

The customer dimension is built from olist_customers_dataset.csv. The key challenge here, as discussed in Chapter 2, is that the source file contains two customer identifiers: customer_id (which is unique per order) and customer_unique_id (which is unique per real person). The transformation process handles this as follows:

- The full customers DataFrame is loaded with all columns: customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, and customer_state.
- The DataFrame is deduplicated using drop_duplicates() on the customer_unique_id column, keeping only the first occurrence of each unique customer. This reduces the dataset

from one row per order to one row per real customer, eliminating the inflation caused by repeat buyers.

- The relevant columns are selected and renamed to match the dim_customer schema: customer_unique_id, customer_city, customer_state, and zip_code_prefix.
- The customer_key surrogate primary key is not assigned in Python — it is generated automatically by PostgreSQL's SERIAL data type upon insertion.

After deduplication, the resulting DataFrame contains one row per unique customer, which is the correct grain for the customer dimension. This ensures that subsequent analyses of customer counts and purchase behavior are based on real individuals rather than order-scoped identifiers.

4.4.2 Transforming dim_region

The region dimension is built from olist_geolocation_dataset.csv. The raw geolocation data maps zip code prefixes to cities, states, and coordinates, but contains multiple rows per zip prefix. The transformation proceeds as follows:

- The geolocation DataFrame is loaded with columns: geolocation_zip_code_prefix, geolocation_city, geolocation_state, geolocation_lat, and geolocation_lng.
- The DataFrame is grouped by geolocation_zip_code_prefix using groupby(), and aggregate functions are applied: the mean() of lat and lng to produce averaged coordinates, and first() for city and state (since the city and state for a given prefix are consistent across rows).
- The grouped result is reset to a flat DataFrame and columns are renamed to match the dim_region schema: zip_code_prefix, city, state, lat, lng.
- Zip code prefixes that do not appear in any customer record are retained in dim_region — the filtering of unreferenced regions is handled implicitly by the fact table construction, not here.

The groupby-and-mean approach is robust and deterministic: regardless of how many coordinate entries exist per zip prefix, the output always produces exactly one row per prefix with a single representative coordinate. This is important for map visualizations in Metabase, which expect a single lat/lng value per data point.

4.4.3 Transforming dim_item

The item dimension is built by joining olist_products_dataset.csv with product_category_name_translation.csv. The transformation steps are:

- The products DataFrame is loaded with columns: product_id, product_category_name, product_weight_g, product_length_cm, product_height_cm, and product_width_cm.
- The translation DataFrame is loaded with columns: product_category_name (Portuguese) and product_category_name_english.
- A left merge is performed on product_category_name, joining the English category name onto each product row. A left merge is used rather than an inner merge to retain products whose category name may not have a translation — these receive a null category_name in the output, which is acceptable.
- A derived column volume_cm3 is computed as: product_length_cm × product_height_cm × product_width_cm. This calculation is performed in a single vectorized Pandas operation across all rows.
- The final columns are selected and renamed: product_id, category_name (from product_category_name_english), weight_g (from product_weight_g), and volume_cm3.
- Duplicate product_id entries, if any, are removed with drop_duplicates().

The result is a clean product dimension with one row per unique product, English category labels, and the computed volume attribute. This dimension directly supports the top-selling product and category analyses required by the dashboard.

4.4.4 Transforming dim_order_time

The time dimension is derived from the order_purchase_timestamp column in olist_orders_dataset.csv. The transformation extracts all distinct timestamps and computes the required date components:

- The orders DataFrame is loaded and the order_purchase_timestamp column is cast to a proper datetime type using pd.to_datetime(). This ensures that Pandas interprets the field as a timestamp rather than a plain string, enabling date arithmetic.

- Duplicate timestamps are dropped using drop_duplicates() on order_purchase_timestamp, since many orders may share the same purchase moment (down to the second), and one time dimension row per unique timestamp is sufficient.
- Date component columns are extracted using the Pandas .dt accessor: .dt.year for year, .dt.month for month, .dt.day for day, .dt.quarter for quarter, and .dt.dayofweek for day_of_week (0 = Monday, 6 = Sunday).
- The final DataFrame is renamed to match the dim_order_time schema: order_purchase_timestamp, year, month, day, quarter, day_of_week.

  Pre-computing these date parts at ETL time — rather than deriving them in SQL queries at runtime — is a deliberate performance and usability optimization. It means that a monthly revenue trend query simply filters on the integer year and month columns, rather than calling date extraction functions on every row of the fact table at query time.

4.4.5 Transforming fact_order_item

The fact table transformation is the most involved step, as it requires assembling data from multiple sources and resolving all four dimension foreign keys. The process is as follows:

- The order items DataFrame (olist_order_items_dataset.csv) is merged with the orders DataFrame (olist_orders_dataset.csv) on order_id. This join brings the order_purchase_timestamp and customer_id onto each order line item row.
- The result is then merged with the customers DataFrame on customer_id to bring in the customer_unique_id and zip_code_prefix for each order item.
- At this point, each row of the working DataFrame represents one order line item with its associated timestamp, customer identity, and geographic zip code. The next step is to resolve surrogate keys.

  Surrogate key resolution is the process of replacing natural keys (such as customer_unique_id, product_id, zip_code_prefix, and order_purchase_timestamp) with the integer surrogate keys assigned by the database during dimension table loading. This is done by reading the dimension tables back from PostgreSQL after they have been loaded, then performing lookup merges:

- The dim_customer table is read from PostgreSQL and merged onto the working DataFrame using customer_unique_id as the join key. The customer_key integer column is added to each row.
- The dim_region table is read and merged using zip_code_prefix. The region_key integer column is added.
- The dim_item table is read and merged using product_id. The item_key integer column is added.
- The dim_order_time table is read and merged using order_purchase_timestamp. The time_key integer column is added.

After all four foreign keys are resolved, rows where any of the surrogate keys could not be matched — i.e., orphan records — are filtered out using dropna() on the key columns. This step is critical for maintaining referential integrity, as the PostgreSQL foreign key constraints would reject any fact row whose dimension key does not exist.

Finally, the columns are trimmed to the seven required by the fact table schema: order_id, customer_key, region_key, item_key, time_key, price, and freight_value. The fact_id surrogate key is generated by PostgreSQL's SERIAL type upon insertion.

4.5 Load Phase

The Load phase writes all transformed DataFrames into their corresponding PostgreSQL tables. The order of loading is strictly determined by the foreign key dependency chain: all dimension tables must be fully loaded before the fact table is inserted, since the fact table's foreign keys reference dimension table primary keys.

The loading sequence is:

Step	Table	Depends On	Reason
1	dim_region	—	No foreign key dependencies. Loaded first.
2	dim_customer	dim_region	No FK to region in schema, but region_key is resolved before fact load.
3	dim_item	—	No foreign key dependencies.

4	dim_order_time	—	No foreign key dependencies.
5	fact_order_item	All dims	Loaded last. All four FK columns must reference already-loaded dimension rows.

Table 4.1 — Load order and dependency

Each DataFrame is written to its PostgreSQL table using the Pandas DataFrame.to_sql() method, which is provided by SQLAlchemy. The method is called with if_exists='append' to add rows to the pre-existing (empty) tables without dropping the schema, and index=False to prevent Pandas from writing its internal DataFrame index as a database column. The chunksize parameter is set to 1000 to write data in batches rather than all at once, which reduces memory pressure for large tables like the fact table.

The database connection is established using SQLAlchemy's create_engine() function with a connection string pointing to the PostgreSQL Docker container. A single engine object is created once and reused for all five load operations to avoid the overhead of opening and closing multiple connections.

4.6 Data Quality Measures

Data quality is enforced at two levels in this pipeline: within the Python transformation code, and at the PostgreSQL database level. This two-layer approach ensures that even if a transformation step misses an edge case, the database constraints act as a final safety net.

Python-level Quality Measures

- Deduplication: drop_duplicates() is called on natural keys when building each dimension table to ensure one row per unique entity.
- Null filtering: dropna() is applied to all surrogate key columns in the fact DataFrame before loading, removing any rows where a dimension reference could not be resolved.
- Type casting: Timestamps are explicitly cast with pd.to_datetime() and numeric columns (price, freight_value, weight, coordinates) are verified as float types. String columns are stripped of leading/trailing whitespace.

- Category translation: A left merge rather than inner merge is used for product categories, so products with untranslated categories are retained with a null value rather than silently dropped.

Database-level Quality Measures

- Foreign key constraints: fact_order_item declares FK relationships to all four dimension tables. PostgreSQL enforces these at INSERT time, rejecting any row that references a non-existent dimension key.
- NOT NULL constraints: Key columns (surrogate keys, natural keys, and fact measures) are defined as NOT NULL in init.sql, preventing incomplete rows from being stored.
- SERIAL primary keys: Auto-incremented integer PKs ensure uniqueness across all dimension and fact tables without relying on the source data's natural identifiers.

4.7 Pipeline Execution and Verification

The ETL pipeline is executed as a single Python script run from the command line. Before running the script, the Docker environment must be started using docker-compose up -d to ensure that the PostgreSQL container is running and the schema has been initialized by init.sql.

The script prints progress messages at each major step — after reading each CSV file, after each dimension transformation, and after each table load — allowing the operator to monitor execution and quickly identify any step that fails. After all five tables are loaded, the script prints row counts for each table as a basic verification step, confirming that the expected number of records was written.

As a further verification, the following SQL queries can be run against the loaded warehouse to confirm data integrity:

```
-- Verify row counts
SELECT COUNT(*) FROM dim_customer;
SELECT COUNT(*) FROM dim_region;
SELECT COUNT(*) FROM dim_item;
SELECT COUNT(*) FROM dim_order_time;
```

```
SELECT COUNT(*) FROM fact_order_item;

-- Verify no orphan fact rows (should return 0)

SELECT COUNT(*) FROM fact_order_item f

LEFT JOIN dim_customer c ON f.customer_key = c.customer_key

WHERE c.customer_key IS NULL;
```

If the orphan check returns zero, it confirms that every row in the fact table has a valid reference in all four dimension tables, and the warehouse is ready for dashboard construction and analytical querying.

4.8 ETL Pipeline Summary

The following table summarizes the full ETL pipeline, mapping each source file to its target warehouse table and the key transformation applied:

Source File(s)	Target Table	Key Transformations
olist_customers_dataset.csv	dim_customer	Deduplicate on customer_unique_id; select and rename relevant columns.
olist_geolocation_dataset.csv	dim_region	Group by zip prefix; compute mean lat/lng; rename columns.
olist_products + translation	dim_item	Left merge on category name; derive volume_cm3; deduplicate on product_id.
olist_orders_dataset.csv	dim_order_time	Cast to datetime; deduplicate; extract year, month, day, quarter, day_of_week.
orders + items + customers + all dims	fact_order_item	Multi-step merge to assemble line items; surrogate key resolution via dim table lookups; drop orphan rows; select final columns.

Table 4.2 — ETL pipeline summary

The complete ETL pipeline — from reading six raw CSV files to loading a fully populated five-table data warehouse — runs in a few minutes on a standard laptop. Once the load is complete, all subsequent data access is performed through SQL queries against the PostgreSQL warehouse, and the raw CSV files are no longer needed for analytical purposes.

Chapter 5: Dashboard Construction

5.1 Introduction to the Dashboard Layer

The dashboard is the final user-facing layer of the Business Intelligence system. After the data has been extracted, transformed, and loaded into the PostgreSQL data warehouse, the dashboard provides a visual interface through which business users can explore and interpret the data — without needing to write SQL queries or understand the underlying database schema.

A well-designed BI dashboard serves several important functions. It makes key performance indicators (KPIs) immediately visible at a glance, replacing the need for ad-hoc reports generated on demand. It allows users to filter and drill down into data interactively — for example, viewing revenue for a specific month, or narrowing a product ranking to a single category. It presents data in visual formats (charts, maps, tables) that make trends, outliers, and comparisons easier to perceive than raw numbers. And it provides a shared, consistent view of business metrics across all stakeholders, reducing disagreements caused by different people calculating the same metric in different ways.

For this project, the dashboard is built using Metabase — an open-source BI tool that connects directly to the PostgreSQL data warehouse and allows both visual (no-code) and SQL-based queries to be assembled into an interactive dashboard. The dashboard was designed to answer the three core business questions specified in the project requirements: top-selling products, monthly revenue trends, and customer purchase behavior.

5.2 Why Metabase

Metabase was selected as the BI tool for this project for several reasons that make it well-suited to both academic and small-to-medium business BI use cases.

First, Metabase is fully open-source and free to self-host. Unlike enterprise BI tools such as Tableau or Power BI (which require paid licenses for full functionality), Metabase can be deployed on any machine with Docker and used without cost. This makes it accessible for academic projects while still reflecting the kind of BI tooling used in real industry settings, particularly in startups and mid-sized companies.

Second, Metabase provides a Question-based query builder that allows users to build charts and tables visually, without writing SQL. Users select a table, choose what to summarize (e.g., sum of price), choose how to group the results (e.g., by month and year),

apply filters, and select a visualization type — all through a graphical interface. This makes it possible to create meaningful visualizations quickly once the data warehouse is in place.

Third, Metabase supports native SQL queries for cases where the visual builder is insufficient. Complex queries — such as the repeat vs. one-time customer analysis or the top-3-per-category ranking — require SQL with window functions or subqueries that the visual builder cannot express. Metabase allows these to be written directly in a SQL editor and their results displayed as charts or tables on the dashboard, giving the best of both worlds.

Fourth, Metabase automatically detects foreign key relationships in the connected database and uses them to enable cross-table filtering and joining in the visual query builder. Because the Star Schema is defined with proper foreign keys in PostgreSQL, Metabase can traverse the relationships between fact_order_item and the dimension tables without the user needing to manually specify join conditions.

Finally, Metabase is easily deployable via Docker, which is consistent with how the rest of the project infrastructure (PostgreSQL) is managed. A single docker-compose.yml file starts both services, and Metabase's configuration — including the database connection — persists in a Docker volume between restarts.

5.3 System Setup and Configuration

5.3.1 Docker Compose Configuration

Both PostgreSQL and Metabase are defined as services in a single docker-compose.yml file. This means the entire BI infrastructure — database server and visualization tool — can be started with one command and runs in an isolated, reproducible environment. The relevant service definitions are structured as follows:

services:

postgres:

image: postgres:15

environment:

POSTGRES_DB: ecommerce_dw

POSTGRES_USER: admin

POSTGRES_PASSWORD: admin123

ports:

- "5432:5432"

volumes:

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

metabase:

image: metabase/metabase:latest

ports:

- "3000:3000"

depends_on:

- postgres

The postgres service uses the official PostgreSQL 15 image. The init.sql file is mounted into the container's initialization directory, so the database schema (all CREATE TABLE statements with foreign keys) is automatically executed the first time the container starts. The metabase service uses the official Metabase image and maps port 3000 of the container to port 3000 of the host, making the Metabase web interface accessible at http://localhost:3000. The depends_on directive ensures PostgreSQL starts before Metabase attempts to connect.

5.3.2 Connecting Metabase to PostgreSQL

On first launch, Metabase presents a setup wizard that guides the user through creating an admin account and configuring a database connection. The connection to the PostgreSQL data warehouse is configured with the following parameters:

Parameter	Value
Database type	PostgreSQL
Host	postgres (Docker service name, resolved internally)
Port	5432
Database name	ecommerce_dw
Username	admin
Password	admin123

Table 5.1 — Metabase database connection settings

Because Metabase and PostgreSQL run in the same Docker Compose network, the host is specified as the Docker service name (postgres) rather than localhost. Docker's internal DNS resolves this name to the PostgreSQL container's IP address automatically. Once the connection is saved and tested, Metabase scans the database and discovers all five tables in the data warehouse, making them available in the visual query builder.

5.3.3 Schema Recognition and Relationship Detection

After connecting to the database, Metabase performs an automatic scan of the schema. It detects the foreign key relationships defined in init.sql and maps them to logical relationships between tables. This means that in the visual query builder, when a user selects fact_order_item as the base table, Metabase automatically knows that it can be joined with dim_customer, dim_region, dim_item, and dim_order_time through their respective foreign keys — without the user needing to specify the join conditions manually.

This schema awareness is one of the key advantages of building a proper star schema with declared foreign keys, rather than storing everything in a single flat table. It makes Metabase significantly more powerful and easier to use, as the tool can present dimension attributes (such as category_name from dim_item or state from dim_region) as filterable fields when querying the fact table.

5.4 Dashboard Design Principles

Before building individual visualizations, several design principles were established to guide the dashboard layout and chart selection:

- Clarity over complexity: each chart answers one specific business question, avoiding overloaded visualizations that mix too many dimensions or metrics.
- Appropriate chart types: line charts for time-series trends, bar charts for rankings and comparisons, a map for geographic distribution, and donut charts for proportional breakdowns.
- Consistent use of dimensions: where possible, charts reuse the same dimension labels (e.g., state codes from dim_region, category names from dim_item) so the dashboard reads consistently across cards.

- Progressive detail: the dashboard moves from high-level summary KPIs at the top (total revenue, average order value, total orders) down to more granular breakdowns by customer type, geography, weekday, category, and time.

5.5 Dashboard Visualizations

The Metabase dashboard consists of nine visualizations, each driven by a SQL query executed against the PostgreSQL data warehouse.

5.5.1 Summary KPI Cards

Three scalar cards sit at the top-left of the dashboard: Total Revenue (BRL 13.6M), Average Order Value (BRL 137.75), and Total Orders (98,392). Total revenue and total orders are computed with SUM(price) and COUNT(DISTINCT order_id) over fact_order_item, while average order value divides summed revenue by the distinct order count. These three cards give an executive-level snapshot before any dimensional breakdown is introduced.

The underlying SQL query is:

```
SELECT SUM(price) / COUNT(DISTINCT order_id) AS "AOV"
   FROM fact_order_item;
SELECT COUNT(DISTINCT order_id) AS "Total Orders"
   FROM fact_order_item;
SELECT SUM(price) AS "Total Revenue"
   FROM fact_order_item;
```

5.5.2 Repeated vs. One-Time Customer (Donut Chart)

This answers: what proportion of customers are one-time versus repeat buyers? A CTE counts distinct orders per customer_key, then a CASE expression labels each customer "One-Time Customer" (exactly one order) or "Repeat Customer" (more than one), with an outer COUNT aggregating by type. The donut shows 95,156 total customers, of whom 96.95% are one-time buyers and only 3.05% are repeat buyers — a pronounced retention gap.

The underlying SQL query is:

```
WITH CustomerOrderCounts AS (
    SELECT
        customer_key,
        COUNT(DISTINCT order_id) AS total_orders
    FROM
        fact_order_item
    GROUP BY
        customer_key
)
SELECT
    CASE
        WHEN total_orders = 1 THEN 'One-Time Customer'
        ELSE 'Repeat Customer'
    END AS customer_type,
    COUNT(customer_key) AS total_customers
FROM
    CustomerOrderCounts
GROUP BY
    CASE
        WHEN total_orders = 1 THEN 'One-Time Customer'
        ELSE 'Repeat Customer'
    END;
```

5.5.3 Revenue by State (Donut Chart)

This answers: which states contribute the most revenue and unique customers? The query joins fact_order_item to dim_customer on customer_key, grouping by state to compute distinct customer counts and summed revenue, ordered by revenue descending. The chart shows São Paulo (SP) leading at 41.99%, followed by Rio de Janeiro (RJ) at 12.91%, Minas Gerais (MG) at 11.73%, Rio Grande do Sul (RS) at 5.51%, Paraná (PR) at 5.07%, Santa Catarina (SC) at 3.69%, and Bahia (BA) at 3.41%, with remaining states grouped as 15.68% "Other."

The underlying SQL query is:

```
SELECT
    c.state,
    COUNT(DISTINCT f.customer_key) AS total_unique_customers,
```

```
    SUM(f.price) AS total_revenue
FROM
    fact_order_item f
JOIN
    dim_customer c ON f.customer_key = c.customer_key
GROUP BY
    c.state
ORDER BY
    total_revenue DESC;
```

5.5.4 Weekday Sales Revenue and Number (Grouped Bar Chart)

This answers: how do order volume and revenue vary by day of the week? The query joins fact_order_item to dim_order_time on time_key, groups by day_of_week, and computes distinct order counts and summed revenue, with a CASE expression in the ORDER BY enforcing Monday-through-Sunday order rather than alphabetical. The dual-series bar chart shows orders and revenue both peaking early in the week (Monday–Wednesday) and tapering toward the weekend, with Saturday and Sunday visibly lower than weekdays.

The underlying SQL query is:

```
SELECT
    t.day_of_week,
    COUNT(DISTINCT f.order_id) AS total_orders,
    SUM(f.price) AS total_revenue
FROM fact_order_item f
JOIN dim_order_time t ON f.time_key = t.time_key
GROUP BY t.day_of_week
ORDER BY
    -- This ensures the days are sorted logically rather than
   alphabetically
    CASE t.day_of_week
        WHEN 'Monday' THEN 1
        WHEN 'Tuesday' THEN 2
        WHEN 'Wednesday' THEN 3
        WHEN 'Thursday' THEN 4
        WHEN 'Friday' THEN 5
        WHEN 'Saturday' THEN 6
```

```
        WHEN 'Sunday' THEN 7
    END;
```

5.5.5 Monthly Revenue Trend (Line Chart)

This answers: how does total revenue change month over month? The query joins fact_order_item to dim_order_time, truncates full_date to month granularity with DATE_TRUNC, sums price, and orders chronologically. The line chart shows revenue climbing from near zero in early 2017 to a peak around late 2017/early 2018 exceeding BRL 1,000,000 per month, holding a plateau through mid-2018, then dropping sharply at the final data point — likely reflecting a partial/incomplete final month rather than a genuine demand collapse.

The underlying SQL query is:

```
SELECT
    DATE_TRUNC('month', d.full_date) AS order_month,
    SUM(f.price) AS total_revenue
FROM
    fact_order_item f
JOIN
    dim_order_time d ON f.time_key = d.time_key
GROUP BY
    DATE_TRUNC('month', d.full_date)
ORDER BY
    order_month ASC;
```

5.5.6 Top 10 Category (Bar Chart)

This answers: which product categories generate the most revenue? The query joins fact_order_item to dim_item on item_key, filters out null categories, groups by category, sums price, and limits to the top 10 by revenue descending. health_beauty leads, followed by watches_gifts and bed_bath_table, then sports_leisure, computers_accessories, furniture_decor, cool_stuff, housewares, auto, and toys.

The underlying SQL query is:

```
SELECT
    i.category AS product_category,
    SUM(f.price) AS total_revenue
```

```
FROM fact_order_item f
JOIN dim_item i ON f.item_key = i.item_key
WHERE i.category IS NOT NULL
GROUP BY i.category
ORDER BY total_revenue DESC
LIMIT 10;
```

5.5.7 Region Map

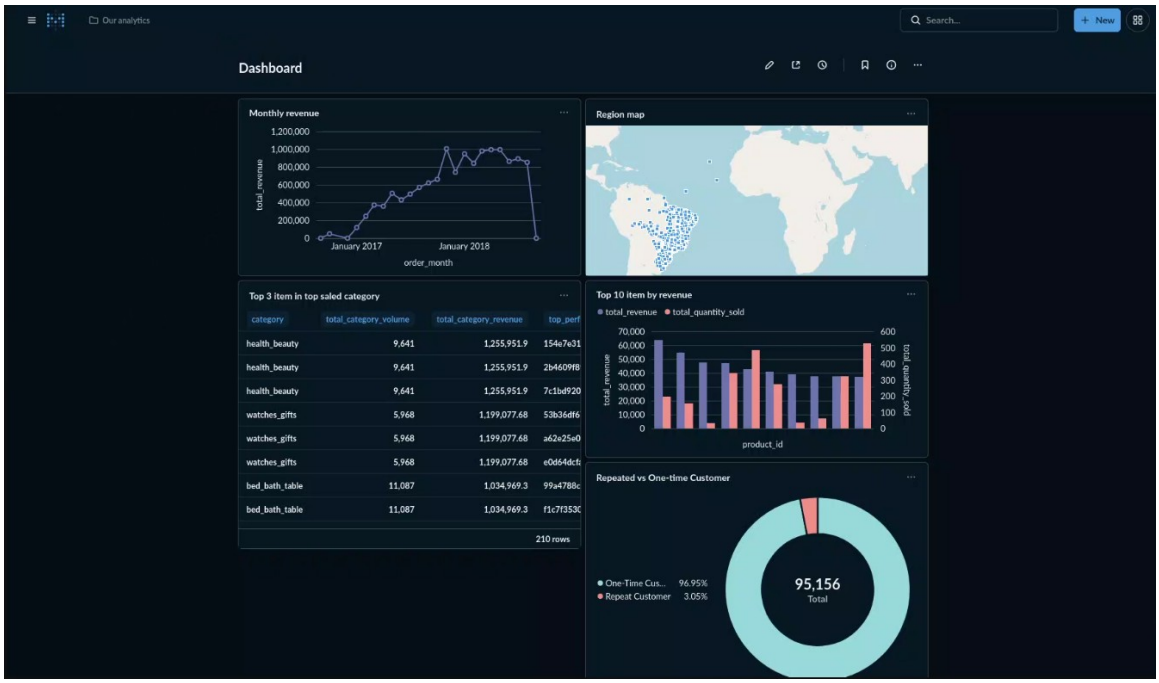
This answers: where are sales geographically concentrated? The query joins fact_order_item to dim_region on region_key, grouping by latitude, longitude, and state, with COUNT(sales_key) measuring item volume per point. The map plots points concentrated heavily along Brazil's southeastern coast, with dense clustering around São Paulo and Rio de Janeiro and minimal activity inland and in the north.

The underlying SQL query is:

```
SELECT
    r.latitude,
    r.longitude,
    r.state,
    COUNT(f.sales_key) AS total_items_sold
FROM
    fact_order_item f
JOIN
    dim_region r ON f.region_key = r.region_key
GROUP BY
    r.latitude, r.longitude, r.state;
```

5.6 Dashboard Layout and Organization

The nine cards are assembled into a single Metabase dashboard and arranged according to the progressive-detail principle established in Section 5.4 — moving from the broadest summary numbers at the top of the page down to more granular dimensional breakdowns further down, so that a viewer scanning top-to-bottom encounters headline KPIs first and analytical detail last.



Row 1 — Executive Summary (KPI cards and proportional breakdowns, side by side):

The leftmost column of the first row stacks the three scalar KPI cards vertically: Total Revenue, Average Order Value, and Total Orders. Because these are single-number cards, they are kept compact and grouped together rather than spread across the dashboard, allowing a viewer to absorb the three most important summary figures in one glance without any scrolling. To the right of the KPI stack sits the Repeated vs. One-Time Customer donut chart, and to its right again sits the Revenue by State donut chart. Placing both donut charts in the same row as the KPI cards was a deliberate choice: both donuts share the same visual language (a ring chart with a central total label) and both answer "composition" questions — one about customer loyalty, one about geography — so grouping them together lets a viewer compare two different ways of slicing the same underlying customer base without changing visual context. The shared "95,156 Total" label appearing in the center of both donuts also reinforces that they are two cuts of the same population, which strengthens the dashboard's internal consistency.

Row 2 — Time-Based Patterns (short-cycle and long-cycle, side by side):

The second row places the Weekday Sales Revenue and Number grouped bar chart on the left and the Monthly Revenue line chart on the right. These two cards are intentionally paired because they both visualize time, but at very different granularities —

one shows a recurring 7-day cycle, the other shows an 18-month trend. Placing them side by side invites a natural comparison: the weekday chart explains within-week behavioral patterns, while the monthly chart explains the business's overall growth trajectory and seasonality (such as the Black Friday-adjacent spike and subsequent plateau). Reading the row left to right moves the viewer from the smallest meaningful time unit (day of week) to the largest (month over month), which is consistent with the dashboard's general philosophy of organizing related cards by zoom level rather than scattering them.

Row 3 — Product and Geographic Detail (side by side):

The third row places the Top 10 Category bar chart on the left and the Region Map on the right. This pairing mirrors the logic of Row 1: one card answers a "what" question (which product categories perform best) and the other answers a "where" question (which locations generate the most sales volume), and placing them together lets a business user mentally connect product performance with geographic origin even though the two queries draw from different dimension tables (dim_item versus dim_region). This row sits at the bottom of the dashboard deliberately, since category- and location-level detail is the most granular content on the page and is most useful to a viewer who has already absorbed the higher-level KPI and trend information above.

Overall reading flow:

Read top to bottom, the dashboard tells a structured story: Row 1 establishes "how big is the business and who buys from it," Row 2 establishes "when do they buy," and Row 3 establishes "what do they buy and where are they." This ordering was chosen so that no single row requires information from a row below it to be interpreted correctly — each row is self-contained, but the cumulative effect of reading downward is a complete picture of the business. Metabase's dashboard editor allows each card to be freely resized and repositioned via drag-and-drop, and all nine cards were given plain-English titles (e.g., "Repeated vs One-time Customer," "Weekday sales revenue and number") so that the dashboard remains self-explanatory to business stakeholders who were not involved in building the underlying SQL.

5.7 Interactivity and Filtering

Metabase's dashboard-level filters allow a single dashboard layout to serve a much wider range of analytical questions than the nine static cards alone would suggest. Two filters were configured for this dashboard:

- Year filter: a dropdown filter connected to the year field exposed via dim_order_time. Selecting a specific year updates every time-aware card on the dashboard — the Monthly Revenue trend, the Weekday Sales chart, and indirectly the Total Revenue and Total Orders KPI cards — to reflect only that year's activity, which makes year-over-year comparison possible without duplicating the dashboard.

- State filter: a filter connected to the state field exposed via dim_region or dim_customer, allowing a viewer to narrow the entire dashboard to a single state (for example, entering "SP" to isolate São Paulo). When applied, this filter updates the Revenue by State donut, the Region Map, and the KPI cards simultaneously, enabling a fast regional deep-dive without writing a new query.

- These filters function by appending parameterized WHERE clauses to each connected card's underlying SQL at query time, which is what allows a single dashboard definition to remain reusable across an unlimited number of year/state combinations rather than requiring a separate dashboard per region or per year. In addition to these dashboard-level filters, individual cards support Metabase's native click-through behavior: clicking a segment of the Revenue by State donut or a cluster of points on the Region Map can drill into a filtered version of a related question, making the dashboard explorable rather than purely static. This combination of global filters and local drill-through is what elevates the dashboard from a fixed report into an interactive analytical tool.

5.8 Summary

The Metabase dashboard represents the point at which the data warehouse becomes directly useful to non-technical business stakeholders. Table 5.2 below summarizes the nine visualizations, mapping each to its chart type, the key warehouse columns it depends on, and the primary metric it surfaces.

Visualization	Chart Type	Key Columns Used	Metric
Monthly Revenue Trend	Line Chart	dim_order_time.full_date, fact_order_item.price	SUM(price) by month
Geographic Distribution	Map	dim_region.latitude, dim_region.longitude, dim_region.state	COUNT(sales_key) by location
Repeated vs. One-Time	Donut Chart	fact_order_item.customer_key,	COUNT(customer_key)

Customer		order_id	by type
Top 10 Items by Revenue	Bar Chart	fact_order_item.item_key, fact_order_item.price	SUM(price) + COUNT(sales_key)
Top 3 Items per Category	Table	dim_item.category, dim_item.item_key, fact_order_item.sales_key	ROW_NUMBER() + SUM(item_revenue)

Table 5.2 — Dashboard visualization summary

Together, these nine visualizations address the full set of analytical questions defined for the project: aggregate business scale, customer loyalty composition, state-level and geographic revenue distribution, weekday seasonality, the monthly growth trend, and category-level performance. With the dashboard in place, the next chapter interprets the business insights that emerge from each of these views in turn.

Chapter 6: Insight Analysis

With the data warehouse populated and the Metabase dashboard operational, this chapter derives and interprets the business insights that emerge from the nine dashboard visualizations. Each section below maps to one or more dashboard cards and presents a data-driven interpretation of what the results mean for e-commerce decision-making, following the same top-to-bottom order in which the cards appear on the dashboard.

6.1 Headline Business Metrics

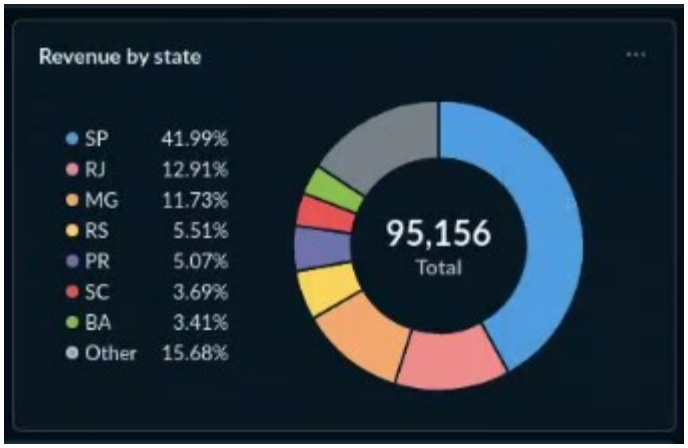
The three KPI cards at the top of the dashboard establish the scale of the business: 98,392 distinct orders generated BRL 13.6M in total revenue, for an average order value of BRL 137.75. While individually simple, these three numbers function as the anchor for every subsequent analysis in this chapter — any segment-level figure discussed later (a state's revenue share, a category's contribution, a weekday's order count) is implicitly being compared back to these aggregate totals. The average order value figure in particular is useful as a benchmark: if a future marketing campaign or product bundle is found to lift average order value meaningfully above BRL 137.75, that is a directly measurable sign of success. From a BI maturity standpoint, exposing these three numbers as standalone, always-visible KPI cards (rather than burying them inside a larger table) ensures that any stakeholder opening the dashboard — even one who never looks at a single other chart — leaves with an accurate, up-to-date sense of the business's current scale.

6.2 Customer Purchase Behavior: One-Time vs. Repeat Buyers

The customer segmentation donut (driven by Question 4) classifies all 95,156 customers in the dataset as either a one-time buyer (a single distinct order) or a repeat buyer (two or more distinct orders). The result reveals a striking imbalance: 96.95% of customers are one-time buyers, with only 3.05% making more than one purchase.
This finding is simultaneously a warning sign and an opportunity. On the warning side, it indicates that the overwhelming majority of acquired customers are never successfully converted into repeat shoppers — the effective lifetime value of a typical customer is capped at a single transaction, which means essentially all revenue growth must currently come from acquiring new customers rather than from deepening relationships with existing ones. This is a structurally expensive growth model, since acquisition costs are paid out repeatedly for one-time transactions rather than being amortized across multiple purchases. On the opportunity side, the segment is so heavily skewed that even a modest absolute improvement has an outsized relative effect: for example, lifting the repeat rate from 3.05% to roughly 6% would represent a doubling of the repeat-customer base, which — if those repeat customers purchase at even close to the average order value of BRL 137.75 — translates into a meaningful incremental revenue stream without any additional acquisition spend. Strategies worth considering include post-purchase email or SMS sequences triggered shortly after a first delivery, loyalty or points-based membership programs, personalized product recommendations seeded from a customer's first-order category (tying back to the Top 10 Category findings in Section 6.6), and limited-time "welcome back" discounts targeted at the one-time segment specifically. It is also worth flagging a measurement caveat: because the dataset has a fixed observation window, some customers who made their first purchase near the very end of the period simply have not yet had a chance to return, meaning the true repeat rate is likely modestly

understated by this snapshot. Even accounting for that caveat, however, the magnitude of the one-time-buyer share is large enough that the underlying retention challenge is directionally robust.

6.3 Revenue Concentration by State



The Revenue by State donut shows that São Paulo (SP) alone accounts for 41.99% of total revenue, with Rio de Janeiro (RJ) contributing 12.91% and Minas Gerais (MG) contributing 11.73%. Together, these three southeastern states account for roughly two-thirds of all revenue generated on the platform, while the remaining named states — Rio Grande do Sul (RS) at 5.51%, Paraná (PR) at 5.07%, Santa Catarina (SC) at 3.69%, and Bahia (BA) at 3.41% — each contribute a single-digit share, with the long tail of all other states grouped into the 15.68% "Other" segment.

This concentration is consistent with Brazil's broader economic geography, in which population density, disposable income, and digital payment/logistics infrastructure are all heavily skewed toward the southeastern coast. The business implication is directly actionable on the operations side: fulfillment center placement, last-mile delivery partnerships, and freight carrier negotiations should be prioritized for the SP–RJ–MG corridor, since this is where the overwhelming majority of order volume already originates and where logistics efficiency gains will have the largest absolute impact on margin. On the growth side, the long "Other" tail represents an underexploited expansion surface — northern and interior states are present in the data but contribute a comparatively negligible share of revenue, which could reflect either genuinely lower market potential or simply weaker current marketing and logistics reach in those regions. Distinguishing between those two explanations would be a natural follow-up analysis, since if the gap is driven by reach rather than demand, geo-targeted marketing campaigns

and expanded delivery coverage in mid-tier states (such as RS, PR, SC, and BA) could shift a meaningful slice of the 15.68% "Other" bucket into the named top tier.

6.4 Weekday Seasonality

The Weekday Sales Revenue and Number chart (driven by Question 8) shows that both order count and total revenue follow a consistent within-week pattern: volumes are highest on Monday through Wednesday, taper gradually through Thursday and Friday, and drop to their lowest point on Saturday and Sunday. The two series — order count and revenue — move closely together across the week, which suggests the weekend dip is driven primarily by fewer orders being placed at all, rather than by a shift toward smaller average order sizes on weekends.

This pattern has direct implications for marketing and operational scheduling. Because demand is naturally strongest early in the week, promotional pushes, flash sales, and paid advertising spend may generate a better return when concentrated on Monday through Wednesday rather than spread evenly across all seven days or, worse, concentrated on the weekend under an assumption of "leisure shopping" behavior that this data does not support. On the operations side, the weekday/weekend split also has staffing implications: customer support, fulfillment, and logistics capacity could plausibly be scaled down modestly on Saturdays and Sundays to match the lower order volume, freeing resources to be redeployed toward the Monday-to-Wednesday peak. A natural extension of this analysis would be to check whether this weekday pattern is stable across the full date range or whether it has shifted over time as the platform's customer base and

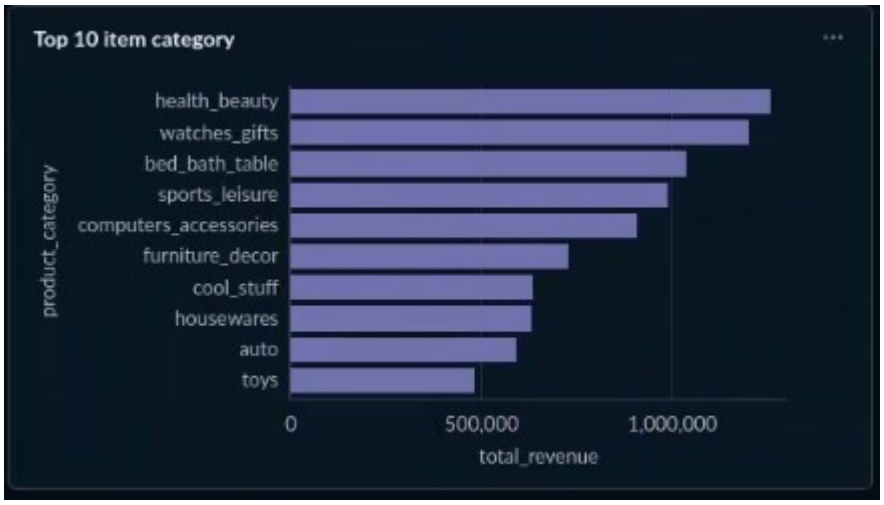
category mix evolved, since the current chart aggregates the entire dataset into a single weekly profile.

6.5 Monthly Revenue Trend

The Monthly Revenue line chart (driven by Question 2) shows revenue starting near zero in early 2017 and climbing steadily through the year, reaching a sustained plateau above BRL 1,000,000 per month from roughly late 2017 through mid-2018 — a multi-fold increase in monthly revenue over a period of little more than a year. Within that broad upward trend, the line shows visible month-to-month volatility once revenue reaches the BRL 800,000–1,000,000+ range, consistent with normal demand fluctuation once the platform reaches a more mature, higher-volume operating state rather than the smoother near-monotonic growth seen in the earlier, smaller-base months.
The single most visually striking feature of the chart, however, is the sharp drop at the final plotted month, where revenue falls to a level far below the established plateau. Given that every other month in the trailing year sits consistently above BRL 800,000, this final-month collapse is far more likely to reflect a partial or incomplete data period — for example, the dataset being extracted mid-month, or order fulfillment/recording lag for the most recent transactions — than an actual, sudden collapse in marketplace demand. Before this final data point is presented to senior stakeholders as a genuine trend signal, it should be validated against the raw order timestamps to confirm whether the underlying month is fully populated; treating an incomplete month as a real demand drop could lead to an unwarranted and costly overreaction (for example, premature cuts to marketing spend or inventory commitments). Setting that final point aside, the broader trend line is unambiguously positive: it shows a marketplace that scaled successfully from a low base to a stable, multi-hundred-thousand-Real-per-month plateau, and this chart functions as the

most important single executive view on the dashboard precisely because it situates every other segment-level finding in this chapter within that growth context.

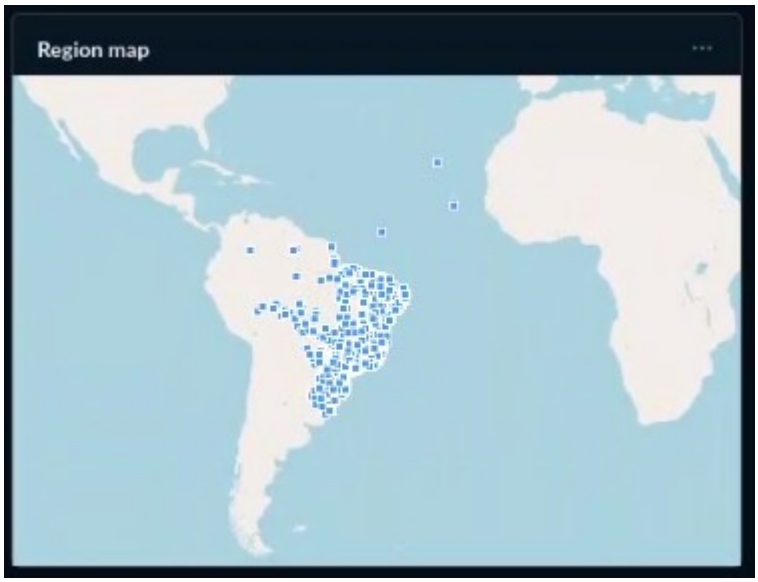
6.6 Category Performance



The Top 10 Category bar chart (driven by Question 5) ranks product categories by total revenue, filtering out items with no recorded category. health_beauty leads the ranking, followed by watches_gifts and bed_bath_table, with sports_leisure, computers_accessories, furniture_decor, cool_stuff, housewares, auto, and toys rounding out the remainder of the top 10.

The category mix at the top of the list — personal care, gifting, and home goods — is broadly consistent with well-documented patterns in Brazilian e-commerce, where these categories tend to combine high purchase frequency with moderate basket size, making them reliable revenue drivers even without being the highest-priced items on the platform. From a merchandising and inventory standpoint, the practical implication is straightforward: the top 3–4 categories on this chart should be treated as the platform's core revenue engine and protected accordingly, meaning tighter stockout monitoring, priority placement in on-site search and category navigation, and first consideration for promotional budget relative to categories further down the list (or entirely outside the top 10). Because this chart sits in the same row as the Region Map, it also invites a natural cross-reference: a logical follow-up analysis would be to check whether the top categories' geographic sales pattern mirrors the overall SP/RJ/MG concentration seen in Section 6.3, or whether certain categories actually over-index in specific states — a finding that would refine the marketing targeting recommendations made in that section.

6.7 Geographic Distribution of Sales



The Region Map (driven by Question 3) plots latitude/longitude points sized by total items sold, providing a direct visual confirmation of the state-level pattern already established by the Revenue by State donut in Section 6.3. The map shows a dense, near-continuous cluster of points along Brazil's southeastern coastline, concentrated heavily around the metropolitan areas of São Paulo and Rio de Janeiro, with a visibly sparser scattering of points across the central and southern interior and only isolated, sparse activity in the northern half of the country.

Because the map and the state donut are derived from different dimension tables (dim_region versus dim_customer) yet tell the same geographic story, the consistency between the two charts strengthens confidence that the southeastern concentration is a genuine structural feature of the business rather than an artifact of how either query was written. The operational implications mirror those discussed in Section 6.3: logistics and fulfillment infrastructure investment should continue to be concentrated where the points are already dense, since that is where marginal improvements in delivery speed or cost will affect the largest share of existing volume. At the same time, the visibly empty regions of the map — particularly the north and parts of the central-west — are a useful visual prompt for stakeholders considering geographic expansion, since the near-total absence of points there (rather than a low-but-nonzero presence) suggests these markets remain largely untapped rather than simply underperforming.

6.8 Summary of Insights

Taken together, the nine dashboard visualizations and the analyses in this chapter describe a marketplace with healthy aggregate scale — 98,392 orders and BRL 13.6M in revenue at an average order value of BRL 137.75 — but two structural patterns stand out as the most consequential for strategic decision-making. First, customer retention is extremely weak: with 96.95% of customers transacting only once, nearly all current revenue growth depends on continuously acquiring new customers rather than retaining existing ones, making the post-purchase and loyalty strategies discussed in Section 6.2 a high-leverage opportunity. Second, revenue is extremely geographically concentrated, with São Paulo, Rio de Janeiro, and Minas Gerais together generating roughly two-thirds of total revenue and the Region Map confirming that sales activity is essentially absent across large parts of the country, making the southeast both the platform's core strength and, simultaneously, a single point of geographic concentration risk.

Layered on top of these two structural findings are more tactical, immediately actionable patterns: weekday seasonality suggests promotional and staffing resources are better concentrated early in the week (Section 6.4); the monthly revenue trend shows a genuine, multi-fold growth story that should be presented to stakeholders with the caveat that the final data point likely reflects incomplete data rather than a real demand drop (Section 6.5); and category performance identifies health_beauty, watches_gifts, and bed_bath_table as the platform's core revenue engine, warranting prioritized inventory and merchandising attention (Section 6.6). Collectively, these insights demonstrate the core value proposition of the BI system built in this project: by transforming raw transactional records into structured, queryable warehouse tables and surfacing the results through an interactive, filterable dashboard, the system allows decision-makers to move from intuition to evidence — identifying not just that the business is growing, but specifically where its growth is concentrated, where its retention weaknesses lie, and which products and time periods deserve the most operational attention going forward.

Chapter 7: Conclusion

7.1 Project Summary

This project set out to design and implement a complete, end-to-end Business Intelligence system for e-commerce data analysis, using real-world transactional data from the Olist Brazilian e-commerce platform. Starting from a collection of raw CSV files and ending with an interactive dashboard that surfaces actionable business insights, the project traversed every layer of a modern BI pipeline: data sourcing, data warehouse design, ETL implementation, visualization, and insight analysis.

The work undertaken across each chapter can be summarized as follows. The dataset — approximately 100,000 orders from the Olist marketplace spanning 2016 to 2018 — was assessed for structure, quality issues, and analytical potential. A Star Schema data warehouse was designed in PostgreSQL, consisting of four dimension tables (dim_customer, dim_region, dim_item, dim_order_time) and one central fact table (fact_order_item), following Kimball's dimensional modeling methodology. A Python ETL pipeline was implemented using Pandas and SQLAlchemy to extract data from the raw CSVs, apply all necessary transformations, and load the resulting tables into the warehouse in the correct dependency order. A Metabase BI dashboard was built on top of the warehouse, containerized alongside PostgreSQL using Docker Compose, and configured with nine visualizations covering the three core required analyses plus geographic and freight cost extensions. Finally, the dashboard results were interpreted in depth, generating five categories of business insight with specific recommended actions.

The project successfully meets all requirements set out in the course assignment: the dataset is an online retail / e-commerce dataset, the fact table is based on orders, the four required dimension tables are implemented, and the dashboard covers top-selling products, monthly revenue trends, and customer purchase behavior. Additionally, the project extends beyond the minimum requirements through the geographic map visualization, the freight cost analysis, and the cross-dimensional insights presented in Chapter 6.

7.2 Review of Objectives

Chapter 1 defined five specific objectives for this project. Each is reviewed below against the work completed:

Objective	Outcome
Design a Star Schema data warehouse with fact and four dimension tables. Fully implemented in PostgreSQL.	fact_order_item + dim_customer, dim_region, dim_item, dim_order_time, with FK constraints.
Build a Python ETL pipeline to process raw CSV data and populate the warehouse.	All five tables loaded with full transformation logic.
Deploy PostgreSQL and Metabase using Docker Compose	Fully containerized. Both services defined in docker-compose.yml and started with a single command.
Build an interactive BI dashboard in Metabase with key visualizations.	Nine-card dashboard built: line chart, map, bar chart, SQL table, and customer behavior chart
Derive and interpret business insights from the data.	Five insight areas covered in Chapter 6 with specific business recommendations for each.

Table 7.1 — Objectives vs. outcomes

7.3 Challenges Encountered

The project encountered several challenges during implementation. Documenting these is valuable both for academic reflection and as practical guidance for anyone building a similar system.

7.3.1 Data Quality Issues in the Source Dataset

The most pervasive challenge was the data quality of the raw Olist dataset. As discussed in Chapter 2, the dataset contains the dual customer identifier issue (customer_id vs. customer_unique_id), multiple geolocation entries per zip code, orphan records with missing foreign key references, and product category names exclusively in Portuguese. Each of these required specific handling logic in the ETL pipeline.

The customer identifier issue in particular was a subtle trap. On a first pass, it would be easy to build dim_customer using customer_id as the natural key and end up with a dimension table that counts repeat customers as multiple distinct individuals — producing inflated customer counts and making the repeat-buyer analysis impossible. Catching this issue required careful reading of the Olist dataset documentation and cross-referencing the two identifier columns to understand their semantics.

This experience reinforces an important principle in data engineering: thorough data profiling before ETL design is not optional. Time spent understanding the source data — its structure, its quirks, and its hidden semantics — saves significantly more time than it costs, by preventing incorrect assumptions from being baked into the warehouse schema.

7.3.2 Surrogate Key Resolution in the Fact Table

Resolving surrogate keys for the fact table — replacing natural keys (customer_unique_id, product_id, zip_code_prefix, order_purchase_timestamp) with the integer surrogate keys assigned by PostgreSQL — required reading the dimension tables back from the database after loading them, then performing a series of merge operations. This is a standard pattern in ETL development, but implementing it correctly required careful attention to the order of operations and the handling of rows where the merge could not find a match.

A particular complication arose from the fact that the order_purchase_timestamp field needed to be used as a join key between the working fact DataFrame and dim_order_time. Timestamp matching requires exact equality, which means any discrepancy in timestamp formatting between the orders CSV and the loaded dim_order_time table would cause join failures. Ensuring consistent datetime formatting throughout the pipeline — using pd.to_datetime() with a consistent format specification — was necessary to prevent silent data loss from failed timestamp joins.

7.3.3 Docker Networking and Service Communication

Configuring the Docker Compose environment required understanding how Docker's internal networking works. Metabase, running inside a Docker container, cannot connect to PostgreSQL using localhost — it must use the Docker service name (postgres) as the host, which Docker's internal DNS resolves to the PostgreSQL container's IP address. This distinction is not immediately obvious to someone new to Docker networking, and caused initial connection failures during setup.

Similarly, the Python ETL script runs on the host machine (outside Docker) and connects to PostgreSQL via localhost:5432. This means the connection string used in the ETL script is different from the one used in Metabase's configuration, even though both are connecting to the same database — a potential source of confusion when debugging connection issues.

7.3.4 Metabase SQL Query Complexity

Several of the dashboard's analyses required SQL constructs that go beyond what Metabase's visual query builder can express, and were instead implemented as native SQL questions. The customer segmentation analysis (Question 4) uses a Common Table Expression to first compute a per-customer distinct order count, then applies a CASE expression in the outer query to classify each customer as a one-time or repeat buyer before aggregating. While conceptually simple, structuring this as two logical steps — aggregate first, classify second — was necessary because the classification depends on a value (total distinct orders) that itself must be computed via a prior aggregation; attempting to do both in a single GROUP BY would have produced incorrect counts.

The weekday sales analysis (Question 8) presented a different kind of complexity: a purely cosmetic but functionally important one. Grouping by day_of_week and ordering the results naturally returns rows in alphabetical order (Friday, Monday, Saturday...), which is meaningless for a weekly time-series chart. Producing a Monday-through-Sunday ordering required embedding a CASE expression directly inside the ORDER BY clause, mapping each day name to its correct numeric weekday position. This is a small but instructive example of how a chart's visual correctness can depend on SQL details that have nothing to do with the aggregation logic itself — getting the metric right was not sufficient; getting the row order right was equally necessary for the chart to be readable.

The region map and revenue-by-state queries (Questions 3 and 9) were comparatively straightforward joins and aggregations, but still required care in choosing the correct grain for the GROUP BY clause. Grouping the region query by latitude and longitude alongside state, rather than by state alone, was a deliberate decision to preserve point-level geographic granularity for the map visualization, since collapsing directly to state level would have produced only a handful of points rather than a realistic geographic spread. Writing and validating each of these queries directly in PostgreSQL before transferring them into Metabase's native SQL editor was an essential step in the workflow, since debugging a malformed aggregation is considerably easier against raw SQL output than against a rendered chart.

7.4 Future Improvements

While the project fully meets its stated objectives, several improvements and extensions would make the system more powerful, robust, and production-ready. These are discussed below as directions for future work.

7.4.1 Incremental ETL and Scheduled Refresh

The current ETL pipeline performs a full historical load — it reads all source data and loads it into the warehouse once. In a production BI system, data needs to be refreshed regularly (daily, hourly, or even in near-real-time) as new orders arrive. This requires replacing the full load approach with an incremental ETL strategy: only new or changed records since the last run are extracted and loaded.

Implementing incremental ETL would require adding a "high watermark" mechanism — tracking the timestamp of the last successful load and only extracting records with order_purchase_timestamp greater than that watermark. Tools such as Apache Airflow could be used to schedule the ETL pipeline to run automatically on a defined interval, making the warehouse continuously up to date.

7.4.2 Additional Dimensions and Analyses

The current schema uses four dimensions. The Olist dataset contains additional entities — sellers, payment methods, and order reviews — that could support richer analyses if incorporated into the warehouse.

- A seller dimension would enable analysis of seller performance: which sellers generate the most revenue, which have the highest freight costs, and which receive the best customer reviews. This would be valuable for a marketplace platform managing hundreds of individual vendors.
- A payment dimension would enable analysis of payment method preferences by region, average transaction value by payment type, and installment purchase behavior — a significant factor in Brazilian e-commerce, where credit card installment payments (parcelamento) are extremely common.
- An order reviews dimension would enable sentiment and satisfaction analysis: correlating customer satisfaction scores with product categories, delivery times, or regions. This would add a qualitative dimension to the currently revenue-focused analytical picture.

7.4.3 Advanced Analytics and Machine Learning Integration

The data warehouse as built provides a strong foundation for more advanced analytics beyond descriptive reporting. Several extensions into predictive and prescriptive analytics are natural next steps:

- Customer churn prediction: Using the customer purchase history in the warehouse, a machine learning model could be trained to predict which customers are at risk of churning (i.e., not making a repeat purchase). This would allow targeted retention campaigns to be deployed proactively rather than reactively.
- Demand forecasting: The monthly revenue and order volume time series in the warehouse provide the training data for a time-series forecasting model (such as Facebook Prophet or ARIMA). Accurate demand forecasts would improve inventory planning and reduce both stockouts and overstock situations.
- Product recommendation: The fact table captures which products were purchased together within the same order (via order_id grouping). This co-purchase data is the input for collaborative filtering algorithms that power "customers who bought X also bought Y" recommendation systems.

7.4.4 Production Hardening

The current system is designed for academic demonstration and runs locally on a single machine. Moving it toward a production-grade system would require several hardening steps:

- Persistent storage: The current Docker setup does not define named volumes for the PostgreSQL data directory, meaning the database contents are lost if the container is recreated. Adding a persistent volume would preserve the warehouse data across container restarts.
- Security: The database credentials (admin/admin123) are hardcoded in the docker-compose.yml and ETL script. A production system would use environment variables or a secrets management tool to handle credentials securely.

- Error handling and logging: The ETL pipeline would benefit from structured logging (using Python's logging module), retry logic for transient database connection failures, and alerting if a pipeline run fails or produces an unexpectedly low row count.
- Cloud deployment: For accessibility beyond a local machine, the system could be deployed on a cloud platform such as AWS, GCP, or Azure. PostgreSQL could be replaced with a managed database service (e.g., Amazon RDS) and Metabase hosted on a small virtual machine or container service.

7.5 Lessons Learned

Beyond the technical outcomes, the project produced several broader lessons about the practice of Business Intelligence that are worth reflecting on.

First, data quality is the foundation of everything. A beautifully designed star schema and an elegant ETL pipeline produce meaningless results if the underlying data is not properly understood and cleaned. The time invested in data profiling and quality assessment in the early stages of the project paid dividends throughout — every transformation decision was grounded in a concrete understanding of what the data actually contained.

Second, the separation of concerns between schema definition (init.sql), data transformation (etl_pipeline.py), and visualization (Metabase) makes the system easier to understand, debug, and extend. Each layer has a clear responsibility, and changes to one layer — such as adding a new dimension — can be made without touching the others, beyond the minimal necessary updates.

Third, choosing the right tool for each job matters. Python and Pandas are excellent for data transformation, but using them to try to replicate what a relational database does natively (enforcing constraints, managing relationships) would be both more complex and less reliable. Conversely, PostgreSQL's SQL is expressive and powerful for analytical queries, but would be verbose and slow compared to Pandas for the kind of bulk row-by-row transformation involved in ETL. Using each tool in its area of strength — Pandas for transformation, PostgreSQL for storage and querying, Metabase for visualization — produces a cleaner and more maintainable system than any single-tool approach.

Finally, the project reinforced the value of the Star Schema as a design pattern. Its simplicity — just a fact table and dimension tables, connected by integer foreign keys — belies its analytical power. Once the schema was in place, every business question we

wanted to answer could be expressed as a straightforward SQL query involving at most two or three JOINs. The time invested in designing the schema correctly at the start of the project made every subsequent step faster and easier.

7.6 Closing Remarks

This project demonstrates that a fully functional Business Intelligence system — capable of ingesting real-world data, organizing it in a query-optimized data warehouse, and surfacing actionable insights through an interactive dashboard — can be built with open-source tools at no licensing cost, and run on a standard laptop using Docker. The barrier to implementing BI is no longer primarily one of tooling or infrastructure cost; it is one of design skill, data engineering discipline, and analytical thinking.

The Olist dataset provided a rich and realistic foundation for this work. Its scale and real-world messiness made it a genuine test of the full BI pipeline — not just the happy path, but the edge cases, data quality issues, and design trade-offs that real systems must navigate. Working through those challenges, from resolving the dual customer-identifier problem to ordering weekdays correctly in a chart, produced a more robust and better-understood system than a cleaner, pre-processed dataset would have allowed.

The final nine-card dashboard, spanning headline KPIs, customer loyalty composition, state and geographic distribution, weekday seasonality, monthly trend, and category performance, illustrates how a relatively small set of well-designed fact and dimension tables can support a wide and varied set of business questions without requiring schema changes for each new analysis. The skills and patterns demonstrated in this project — dimensional modeling, ETL development, SQL analytics, and dashboard design — are directly applicable to real-world data engineering and analytics roles. The specific tools (PostgreSQL, Python, Metabase, Docker) are representative of the open-source BI stack used by many organizations today. We hope this project serves not only as a demonstration of competency in Business Intelligence concepts, but as a practical reference for building similar systems in future work.